PROGRAMACIÓN CONCURRENTE Y DISTRIBUIDA
BOLETÍN DE PRÁCTICAS

CONVOCATORIA DE ENERO
CURSO 2025-2026

Contenido

ENTREGABLES **3**
FORMATO DE LOS DOCUMENTOS **3**
EVALUACIÓN **4**
EJERCICIO 1: INTRODUCCIÓN A LA PROGRAMACIÓN CONCURRENTE. 5
EJERCICIO 2: SEMÁFOROS. 6
EJERCICIO 3: MONITORES. 7
EJERCICIO 4: PASO DE MENSAJES. 8

Entregables

La entrega constará de un único proyecto conteniendo 4 paquetes (uno para cada ejercicio) que se comprimirá en un fichero .zip y una memoria de prácticas en formato PDF. Los ejercicios se implementarán en Java 8 o una versión superior usando la codificación de caracteres UTF-8 (Project-Resource-Textfileencoding-Other-UTF-8). **Tanto el código fuente como la memoria se entregarán a través de una tarea que será creada en el AULA VIRTUAL. No se admitirán entregas realizadas por otros medios.**

En la memoria escrita **se deben incluir necesariamente para cada ejercicio** los siguientes apartados:

- **Recursos no compartibles.** Los que se hayan identificado en el problema.
- **Condiciones de sincronización.** Las detectadas en el problema.
- **Pseudocódigo comentado.** El de los diferentes hilos y monitores que intervengan en la solución.
- **Cuestiones planteadas en los ejercicios.**
- **Código Java de los ejercicios comentado.**

Formato de los documentos

La memoria de prácticas deberá estar escrita correctamente en sus aspectos ortográfico y gramatical, evitando el uso de vulgarismos y lenguaje SMS. Las páginas deberán estar numeradas y se debe incluir un índice. El tipo de letra utilizado para el texto normal será Times New Román (o semejante) en 12 puntos y en color negro. El interlineado será sencillo.

Los listados de código deberán estar formateados cuidando varios aspectos:

- Indentación.
- Tamaño de la letra adecuado para que no se corten los renglones, o cuando esto ocurra el código siga siendo legible y ordenado.
- Tipo de letra de paso fijo (Courier o semejante) en color negro.

Además, el código fuente entregado debe estar debidamente comentado tanto en los bloques de código más relevantes en relación con la programación concurrente y distribuida como mediante el uso de Javadoc para las clases y métodos desarrollados.

En la portada de la memoria de prácticas deberá aparecer arriba a la derecha la siguiente información: Apellidos, Nombre, Asignatura, Grupo y Subgrupo de Prácticas, Convocatoria, Año académico y Profesor.

Evaluación

La evaluación constará de dos partes principales: la corrección de las prácticas entregadas y la defensa de las mismas en una entrevista personal. Las prácticas se realizarán en parejas de 2.

Como criterios de evaluación se tendrán en cuenta los siguientes:

- Corrección de las soluciones aportadas.
- Grado de concurrencia
- Entrevista de prácticas
- Claridad en la explicación (memoria de prácticas)
- Originalidad.

La copia de prácticas será motivo de suspenso para todos los participantes implicados. La no asistencia (injustificada) a la entrevista de prácticas será motivo de suspenso de las mismas.

Este boletín se compone de cuatro ejercicios que serán valorados por el profesor con una puntuación de 0 a 10 cada uno. Para superar la parte práctica será necesario sacar una nota igual o superior a 4 en cada uno de ellos. Cuando todos los ejercicios estén superados la ponderación aplicada será la siguiente: 0.25*E1 + 0.25*E2 + 0.25*E3 + 0.25*E4. La fecha límite para la entrega del boletín de prácticas será **el 21 de diciembre a las 23:55 horas.**

Ejercicio 1: Introducción a la programación concurrente.

Supongamos dos hilos con la siguiente funcionalidad: Un primero hilo se dedica a generar una matriz A de valores enteros de tamaño 3x3, calcular el cuadrado de dicha matriz (A^2) e imprimir en pantalla:

A x A

a1	**a2**	**a3**		**a1**	**a2**	**a3**
a4	**a5**	**a6**	**x**	**a4**	**a5**	**a6**
a7	**a8**	**a9**		**a7**	**a8**	**a9**

A^2

r1	**r2**	**r3**
r4	**r5**	**r6**
r7	**r8**	**r9**

El otro hilo hace lo mismo, pero en lugar de calcular A^2, calcula A+A y por lo tanto imprime en pantalla:

A + A

a1	**a2**	**a3**		**a1**	**a2**	**a3**
a4	**a5**	**a6**	**+**	**a4**	**a5**	**a6**
a7	**a8**	**a9**		**a7**	**a8**	**a9**

2A

r1	**r2**	**r3**
r4	**r5**	**r6**
r7	**r8**	**r9**

La actividad anterior la repite cada hilo 10 veces. Desarrollar un programa concurrente en Java que resuelva el problema anterior. Para resolver la sincronización necesaria entre los hilos se usarán los cerrojos de la clase ReentrantLock.

a) (1 punto sobre 10) Si no usaras ningún mecanismo para sincronización ¿Cómo podría ser la salida en pantalla del programa anterior?

Ejercicio 2: Semáforos.

Desarrollar un programa concurrente en Java para resolver el problema siguiente: Supongamos que en el sistema tenemos tres instancias de la clase Panel que se proporciona. Hay 10 hilos cuya funcionalidad es la siguiente: 1) generar aleatoriamente dos matrices de tamaño 10 X 10 (matrices A y B), 2) sumar A y B obteniendo la matriz C y 3) mostrar en un panel que esté disponible la siguiente salida

"Usando panel el hilo id

Matriz A=…

Matriz B=…

Matriz C=…

Terminando de usar panel el hilo id"

donde los puntos representan los valores de las matrices. **Los pasos 1, 2 y 3 los repiten cada hilo tres veces.**

Por lo tanto, los hilos comparten los tres paneles del sistema y usando semáforos se debe de añadir la sincronización necesaria para que los hilos puedan mostrar su salida en pantalla a través de un panel de forma correcta y el uso de los paneles sea eficiente. La sincronización hay que resolverla con semáforos.

Según las condiciones de sincronización del problema,

a) (0.5 puntos sobre 10) ¿Qué problema clásico es el que te ha ayudado a resolver este problema?

b) (0.5 puntos sobre 10) Si consideramos un panel solamente ¿cómo es el acceso al mismo por parte de los hilos? Si consideramos los tres paneles ¿cómo es el acceso por parte de los hilos? Explica ambas situaciones.

c) (0.5 puntos sobre 10) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?

Ejercicio 3: Monitores.

En este ejercicio, vamos a representar mediante un programa concurrente la actividad en un lavadero de coches. El proceso para lavar el coche es el siguiente: Un cliente llega al lavadero, tiene que ir a una máquina de ticket y elegir el servicio que necesita (esta acción tarda X milisegundos en realizarla). El lavadero tiene un total de 3 máquinas, el cliente se tendrá que poner en la primera que encuentre libre, una vez seleccione el servicio que necesita se dirige a la cola con el menor tiempo de espera. El lavadero tiene 4 zonas de lavado posibles, por lo tanto, habrá una cola para cada zona de lavado. Un cliente tarda Y milisegundos en lavar el coche. Usando monitores para la sincronización, haremos una simulación donde habrá 50 hilos clientes con sus valores X, Y inicializados de forma aleatoria.

La impresión en pantalla por parte de los hilos clientes justo antes de ponerse en la cola de la zona de lavado debe tener el siguiente formato:

"Cliente id ha solicitado su servicio en la máquina: -----

Tiempo en solicitar el servicio: X

El coche será lavado en el lavadero nº: -----

Tiempo en lavar el coche= Y

Tiempo de espera en lavadero1=----, lavadero2=-----, lavadero3=-----, lavadero 4=-----"

---

donde id, -----, X, Y, son valores particulares para cada hilo. Los valores de lavadero 1, lavadero 2, lavadero 3 y lavadero 4 son los valores que le llevan al cliente a tomar la decisión de a qué lavadero dirigirse, es decir, sin incluirse él mismo. Los tiempos X e Y se reflejarán mediante el método Sleep() de Java.

Desarrollar un programa concurrente en Java para resolver el problema anterior usando monitores como mecanismo para la sincronización.

Según las condiciones de sincronización del problema,

a) a) (0.5 puntos sobre 10) Indica si la acción de lavar los coches es concurrente o en exclusión mutua justificando la respuesta.

b) b) (0.75 puntos sobre 10) ¿Qué tipo de monitor Java has usado? Justifica la respuesta.

c) c) (0.5 puntos sobre 10) En el monitor diseñado, ¿has usado notify/signal o notifyAll/signalAll? Justifica la respuesta.

d) d) (0.75 puntos sobre 10) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema?

Ejercicio 4: Paso de mensajes.

Consideremos un conjunto de 50 personas que van a comprar entradas y tiene que pagarlas. Para pagar hay dos colas, la cola R y L siendo la cola R más rápida que la L. Cada persona tiene que usar una cola para pagar durante un tiempo estimado (valor aleatorio entre 1 y 10). El controlador de colas, cuando llega una nueva persona a pagar estima el tiempo de pago y a aquellas personas cuyo tiempo estimado sea mayor o igual a 5 se les asignará la cola más rápida, es decir, la R. A las demás personas se les asignará la cola L. Cada persona comprará los tickets y se pondrá en cola para pagar 5 veces, es decir, el hilo repite 5 veces la siguiente secuencia de acciones: 1) Realiza la selección de entradas (representado con una llamada Thread.sleep(tiempoAleatorio)), 2) solicita ponerse en una cola, 3) Realiza el pago 4) libera la cola e 5) imprime en pantalla la información que se indica a continuación:

"Persona id ha usado la cola X
Tiempo de pago= T
Thread.sleep(T)
Persona id liberando la cola X"

---

donde X es la cola asignada y T el tiempo de pagar.

Desarrollar un programa concurrente en Java para resolver el problema anterior usando como mecanismo para la sincronización el paso de mensajes asíncrono.

a) (0.5 puntos sobre 10) ¿Se pueden usar simultáneamente las dos cajas? Justifica la respuesta.

b) (0.5 puntos sobre 10) ¿Cómo has resuelto la exclusión mutua de la pantalla?